

LangChain Debugging — Worked Example

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

One scenario, followed across all 4 rounds: Doc05's `chain.py` — the reusable `prompt | model | parser` extraction chain later documents build on. Every round is a different way the text going into the chain doesn't match what the chain actually expects. Work through the rounds in order. Each ends with **"What do you think is wrong?"** — stop and actually answer before reading on.

- [Round 1: Basic](#)
- [Round 2: Intermediate](#)
- [Round 3: Real-world](#)
- [Round 4: Multi-agent](#)

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

Round: Basic

Setup:

```
prompt = ChatPromptTemplate.from_template("Answer this question: {question}")
chain = prompt | ChatOpenAI() | StrOutputParser()
```

Called from another part of the app:

```
result = chain.invoke({"input": "What is the refund policy?"})
```

Symptoms: fails on every call, immediately, before the model is ever reached — not intermittent.

Error output:

```
Traceback (most recent call last):
  File "project_1_supportdesk_chat_and_triage/qa.py", line 18, in answer
    result = chain.invoke({"input": "What is the refund policy?"})
    ...
KeyError: "Input to ChatPromptTemplate is missing variables {'question'}.
Expected: ['question'] Received: ['input'] ..."
```

Expected vs. actual:

- Expected: `chain.invoke(...)` fills in the prompt template and returns the model's text answer.
- Actual: it fails before ever calling the model — the prompt template can't find the value it needs.

What do you think is wrong?

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

Round: Intermediate

Setup: the invoke-key bug is fixed. The chain now uses a structured Pydantic parser instead of plain text:

```
class Answer(BaseModel):
    summary: str
    confidence: float

prompt = ChatPromptTemplate.from_template("Answer this question: {question}")
parser = PydanticOutputParser(pydantic_object=Answer)
chain = prompt | ChatOpenAI() | parser
```

Symptoms: this looks like it could be an OpenAI problem (the error talks about the model's "completion"), but the API call succeeds every time — the model answers, and the answer even *looks* sensible to a human reading it.

Error output:

```
Traceback (most recent call last):
  File "project_1_supportdesk_chat_and_triage/qa.py", line 24, in answer
    result = chain.invoke({"question": "What is the refund policy?"})
    ...
langchain_core.exceptions.OutputParserException: Failed to parse Answer
from completion {"summary": "Refunds within 30 days.", "confidence": "high"}.
Got: 1 validation error for Answer
confidence
  Input should be a valid number, unable to parse string as a number
```

Expected vs. actual:

- Expected: the parser reads the model's reply and returns a checked `Answer` object, with `confidence` as a number.
- Actual: the model wrote `"confidence": "high"` — a reasonable guess, but not a number. It sometimes writes `0.9`, sometimes `"high"`, sometimes leaves the field out.

What do you think is wrong?

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

Round: Real-world

Setup: the parser bug is fixed. To give each question its own system message, someone changed the code to build the prompt from the user's text on every call:

```
def answer(user_question):
    prompt = ChatPromptTemplate.from_messages([
        ("system", "You answer support questions briefly."),
        ("human", user_question),
    ])
    chain = prompt | ChatOpenAI() | StrOutputParser()
    return chain.invoke({})
```

Symptoms: only sometimes happens — it depends entirely on what text is in the question, not on load or timing. Short, normal questions all work.

Error output (only for certain questions):

```
Traceback (most recent call last):
  File "project_1_supportdesk_chat_and_triage/qa.py", line 31, in answer
    return chain.invoke({})
...
KeyError: 'Input to ChatPromptTemplate is missing variables {\\"status\\"}.
Expected: [\\"status\\"] Received: []
Note: if you intended {\"status\"} to be part of the string and not a
variable, please escape it with double curly braces ...'
```

The failing question was: `Why does my config return {"status": "error"}?`

Expected vs. actual:

- Expected: any question text, including text with `{` and `}` in it (a JSON snippet, a config example), gets sent to the model as-is.
- Actual: works for ordinary questions, and fails exactly when the question contains curly braces. The error's own "Note" suggests escaping the braces.

What do you think is wrong? (And is escaping the braces the real fix, or a symptom fix?)

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

Full solution: [Show me the solution](#)